CAPSTONE / CYBERSECURITY PROJECT

CYBERSHIELD

Automated Web Security Auditor

Project category: Automated Web Scanner (Topic 2)
Technology: Python 3.10+, Streamlit, standard-library HTTP tooling
Deliverables: Runnable app, CLI, source code, offline tests, HTML/JSON reports and GitHub workflow
Release: Version 1.0 | October 2026

A small, transparent security-audit workflow for authorized website owners. The scanner examines observed response headers, HTTPS usage, CORS behavior and cookie attributes, and provides prioritized configuration guidance.

Ethical use: Operate only against websites you own or are authorized to assess. The included demo uses simulated response data; it is not evidence of real-world vulnerabilities.

01 Project overview

Abstract

CyberShield is a Python-based educational web security auditor with an interactive Streamlit dashboard and command-line interface. The tool collects only selected response metadata from a permitted public website, identifies observable configuration gaps, and translates findings into human-readable remediation guidance. Reports can be exported in JSON for further processing and standalone HTML for review or submission.

Problem statement

Website operators often overlook HTTP defense-in-depth configuration or have trouble explaining which settings should be reviewed. Manual checks are repetitive, and many automated scanners are too intrusive for introductory demonstrations. CyberShield provides a small, reproducible alternative with explicit scope and known limitations.

Objectives

- Review HTTPS, HSTS, CSP presence, anti-framing protection, MIME sniffing protection and referrer policy.
- Perform a single limited Origin reflection observation for common CORS mistakes without attempting exploitation.
- Flag cookie attributes while protecting cookie values in generated reports.
- Prioritize review items, create report exports and support repeatable offline unit tests.

Architecture

Layer	Responsibility	Implementation
Interface	Permission confirmation; start audit; review findings	app.py / Streamlit
Transport	Validate URL, resolve public addresses, fetch headers, limit redirects	cybershield/scanner.py
Assessment	Interpret header configuration, CORS and cookie flags	assess_headers(...)
Presentation	Compute heuristic score; export HTML and JSON	cybershield/reports.py
Verification	Run synthetic and mocked tests in CI	tests/ and GitHub Actions

Implementation environment

Language: Python 3.10+; **UI dependency:** Streamlit; **Scanner dependencies:** Python standard library; **Platform:** Windows, macOS or Linux; **Internet:** needed for authorized live audits only. Offline demo and offline tests need no target network access.

02 Methodology and checks

After a user confirms authorization, CyberShield validates the supplied HTTP(S) URL and rejects non-global resolved addresses. It performs a GET request, examines response headers without retaining the response body, follows at most three validated redirects, and may send one additional benign request with a synthetic Origin header. It then produces an evidence-limited assessment and recommendations.

Implemented checks and interpretation

Check	Observed condition	Important caution
Transport	HTTPS on the final response	Not a comprehensive TLS audit
HSTS	Positive max-age on HTTPS; duration	Only the tested response
CSP	Enforced CSP header present	Directive quality not validated
Framing	CSP frame-ancestors or X-Frame-Options	Only response-level evidence
MIME / Referrer	nosniff and Referrer-Policy header	App context still matters
CORS	Arbitrary Origin reflection + credentials	Not proof of data exposure
Cookies	Secure, HttpOnly and SameSite attributes	Some attributes depend on purpose

Severity-based illustrative score

The configuration score starts at 100. A high-severity warning subtracts 17 points, a medium warning subtracts 10 points, and a low warning subtracts 5 points, with a minimum of zero. Informational messages do not reduce the score. Letter grades are A (90-100), B (75-89), C (60-74), D (40-59) and F (0-39). This is an educational prioritization aid, **not** a calibrated security-risk measure.

CORS accuracy note

Credentialed browser access cannot use a wildcard **Access-Control-Allow-Origin: ***. CyberShield treats this combination as a configuration warning rather than a verified credential-exfiltration vulnerability. The strongest CORS warning requires an arbitrary supplied Origin to be echoed and credentialed sharing to be enabled. Even then, exploitability depends on the application and response content.

03 Evaluation and results

Offline test strategy

The included unittest suite uses simulated response snapshots and mocked DNS results. It exercises URL validation, blocking of private network addresses, strong-configuration pass conditions, reflection-based CORS warnings, wildcard-with-credentials interpretation, non-disclosure of cookie values in exports, HTML escaping and accurate demo labeling.

Validation scenario	Expected behavior	Result
Normal URL and unsupported schemes	Normalize allowed URLs; reject invalid ones	PASS
Private vs public destination	Reject loopback; allow global test IP	PASS
Strong header set	No review warnings; indicative score 100	PASS
Arbitrary Origin + credentials	HIGH CORS warning	PASS
Wildcard + credentials	Never claim confirmed credential leak	PASS
Cookie values + HTML input	No secret values; HTML escaped	PASS
Demo report labeling	Clearly distinguish simulations	PASS

Executed result: 13 of 13 offline unit tests passed at build time. These unit tests do not establish compatibility with all real websites or browsers.

Simulated demonstration (not a real target)

The app includes an offline fixture at **https://demo.example/** to illustrate the presentation. It intentionally has a missing HSTS header, missing CSP, missing anti-framing protection, an arbitrary-origin credentialed CORS reflection and a cookie missing review attributes. Other settings are present. These findings are **synthetic** and do not refer to an actual website.

Metric	Simulated value	Interpretation
Score / grade	43 / 100 (D)	Educational scoring only
Review warnings	5	Configuration review priorities
Passed observations	3	Observed protections in fixture
Informational	1	Non-penalized observation

Reproduction commands

Tests: `python -m unittest discover -s tests -v`

Demo: `python -m cybershield.cli --demo` | UI: `streamlit run app.py`

04 Safety, limitations and conclusions

Ethical and operational safeguards

- User confirmation is required before a live scan; use is intended only for owned or explicitly authorized websites.
- Private, loopback and reserved resolved addresses are blocked, and each redirect is checked before following.
- HTTPS certificates are verified by the standard TLS context; response bodies and cookie values are not retained.
- No crawling, authentication bypass, payload exploitation, brute force or state-changing requests are performed.

Known limitations

- DNS validation is best-effort and is not hardened against DNS rebinding. Keep this educational dashboard on localhost.
- One page-level response cannot represent all routes, authenticated features or API permissions.
- CSP presence is not a full CSP assessment; missing headers can be context-dependent and not necessarily exploitable.
- HTTPS checks do not measure certificate expiration, cipher suites or wider TLS configuration.
- The tool does not test BOLA, injection, XSS, CSRF or OWASP Top 10 comprehensively.

Future enhancement opportunities

With stronger authorization controls, the project could gain an allowlisted multi-page audit, richer CSP policy analysis, structured CVSS-aligned triage, an authenticated test harness for self-owned demo applications, and signed machine-readable reports. Public hosting would additionally require robust server-side access restrictions, DNS pinning and network isolation.

Conclusion

CyberShield demonstrates a practical defensive cybersecurity workflow: controlled collection of security-relevant configuration data, transparent interpretation, prioritization of review items, visual reporting and automated verification. The design prioritizes explainability, low setup cost and ethical operation over aggressive scanning. It is appropriate as a student portfolio project when its limited scope and simulated demo are disclosed.

References

1. OWASP - HTTP Security Response Headers Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Headers_Cheat_Sheet.html

2. MDN - Cross-Origin Resource Sharing. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS

3. MDN - Credentialed CORS and wildcard Origin. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS/Errors/CORSNotSupportingCredentials

4. GitHub Docs - Adding a file to a repository. https://docs.github.com/en/repositories/working-with-files/managing-files/adding-a-file-to-a-repository